Python Exercises

Basic concepts hands-on questions

1. Simple Hello World

```
print("Hello World!")
```

2. Jupyter Notebook

```
pip install notebook

jupyter notebook
```

3. VS Code Setup

Install Python extension → create main.py → run using Ctrl+F5

4. Float Precision

```
def calculate_net_salary(salary: float, tax_rate: float) -> float:
    if salary < 0 or not (0 <= tax_rate <= 1):
        raise ValueError("Invalid salary or tax rate value.")
    net_salary = salary * (1 - tax_rate)
    return net_salary

salary_val = 75000.5
tax_rate_val = 0.18
net_sal = calculate_net_salary(salary_val, tax_rate_val)
print(f"Net Salary: {net_sal:.2f}")
```

5. Multiple Assignment

```
def display_coordinates(coords: tuple):
    if not isinstance(coords, tuple) or len(coords) != 2:
        raise ValueError("Coordinates must be a tuple of two elements.")
    if not all(isinstance(n, (int, float)) for n in coords):
        raise TypeError("Coordinates must be numeric.")
    x, y = coords
    print(f"X-Coordinate: {x}, Y-Coordinate: {y}")

point = (10.5, 20.3)
display_coordinates(point)
```

6. Modulo Operator

```
def check_even_odd(number: int):
    if not isinstance(number, int):
        raise TypeError("Number must be an integer.")
    remainder = number % 2
    if remainder == 0:
        print("Even")
    else:
        print("Odd")

number_val = 17
check_even_odd(number_val)
```

7. Floor Division

```
def calculate_share(total_bill: float, people: int) -> int:
    if total_bill < 0 or people <= 0:
        raise ValueError("Bill must be non-negative and people must be greater than zero.")
    share = int(total_bill // people)
    return share


bill_val = 1250
people_val = 4
individual_share = calculate_share(bill_val, people_val)
print(f"Individual Share: {individual_share}")
```

8. Min/Max Functions

```
def find_salary_extremes(salaries: list):
    if not isinstance(salaries, list) or len(salaries) == 0:
        raise ValueError("Salary list cannot be empty.")
    if not all(isinstance(s, (int, float)) for s in salaries):
        raise TypeError("All salaries must be numeric.")
    lowest = min(salaries)
    highest = max(salaries)
    print(f"Lowest Salary: {lowest}")
    print(f"Highest Salary: {highest}")


salary_list = [50000, 75000, 62000, 95000]
find_salary_extremes(salary_list)
```

9. Basic Input

```
def greet_user():
    name = input("Enter your name: ").strip()
    if not name:
        print("Error: Name input cannot be empty.")
        return
    print(f"Hello, {name}!")
```

10. Numeric Input

```
def predict_age():
    age_str = input("Enter your age: ").strip()
    if not age_str.isdigit():
        print("Error: Input must be a valid positive integer.")
        return
    age = int(age_str)
    print(f"Next year you'll be {age + 1}")
```

11. Float Input

```
def kg_to_lbs():
    kg_str = input("Enter weight in kilograms: ").strip()
    try:
        kg = float(kg_str)
        if kg < 0:
            print("Error: Weight cannot be negative.")
            return
    except ValueError:
```

```
        print("Error: Input must be a valid decimal number.")
        return
    lbs = kg * 2.20462
    print(f"Weight in pounds: {lbs:.2f}")
```

12. Simple If

```
def check_passing(marks: int):
    if not isinstance(marks, (int, float)) or not (0 <= marks <= 100):
        raise ValueError("Marks must be a number between 0 and 100.")
    if marks >= 40:
        print("Pass")
    if marks < 40:
        print("Fail")

marks_val = 75
check_passing(marks_val)
```

13. If-Else

```
def check_even_odd_conditional(num: int):
    if not isinstance(num, int):
        raise TypeError("Input must be an integer.")
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")

num_val = 8
check_even_odd_conditional(num_val)
```

14. If-Elif-Else

```
def assign_grade(score: int):
    if not isinstance(score, (int, float)) or not (0 <= score <= 100):
        raise ValueError("Score must be between 0 and 100.")
    if score >= 90:
        grade = "A+"
    elif score >= 80:
        grade = "A"
    elif score >= 70:
        grade = "B"
    elif score >= 60:
        grade = "C"
    else:
        grade = "Fail"
    print(f"Score: {score} -> Grade: {grade}")

score_val = 88
assign_grade(score_val)
```

15. Nested If

```
def login_system(input_user: str, input_pwd: str):
    if not input_user.strip() or not input_pwd.strip():
        print("Error: Username and password cannot be blank.")
        return

    stored_user = "admin"
    stored_pwd = "pass123"

    if input_user == stored_user:
        if input_pwd == stored_pwd:
            print("Access Granted")
        else:
            print("Incorrect Password")
    else:
        print("Username Not Found")

login_system("admin", "pass123")
```

16. For Loop Basics

```
def loop_one_to_five():
    count_limit = 5
    if count_limit <= 0:
        raise ValueError("Count limit must be greater than zero.")
    for i in range(1, count_limit + 1):
        print(i)

loop_one_to_five()
```

17. While Loop

```
def countdown_timer():
    count = 5
    if count <= 0:
        raise ValueError("Initial countdown value must be greater than zero.")
    while count > 0:
        print(count)
        count -= 1

countdown_timer()
```

18. Break Statement

```
def find_first_even_in_range(start: int, end: int):
    if start > end:
        raise ValueError("Invalid range size.")
    for i in range(start, end + 1):
        if i % 2 == 0:
            print(f"First even number found: {i}")
            break

find_first_even_in_range(1, 10)
```

19. Continue Statement

```
def sum_odds_in_range():
    limit = 10
    if limit < 0:
        raise ValueError("Range limit must be non-negative.")
    odd_sum = 0
    for i in range(limit):
        if i % 2 == 0:
            continue
        odd_sum += i
    print(f"Sum of odd numbers: {odd_sum}")


sum_odds_in_range()
```

20. Pass Statement

```
def empty_placeholder_function():
    pass


empty_placeholder_function()
print("Function defined")
```

21. Consistent Indentation

```
def test_nested_indentation(cond1: bool, cond2: bool):
    if cond1 is True:
        if cond2 is True:
            print("Nested")
    print("Indentation verification complete.")

test_nested_indentation(True, True)
```

22. Comment Usage

```
def calculate_total_salary():
    base_salary = 60000
    bonus = 15000
    total = base_salary + bonus
    print(f"Total Salary: {total}")


calculate_total_salary()
```

23. Import Standard Module

```
import math


def calculate_circle_area(radius: float):
    if radius < 0:
        raise ValueError("Radius cannot be negative.")
    area = math.pi * (radius ** 2)
    print(f"Area of circle: {area:.2f}")


calculate_circle_area(5.5)
```

24. All Import

```
from math import *


def demonstrate_math_utilities(value: float):
    if value < 0:
        raise ValueError("Input value must be non-negative for standard calculations.")
    square_root = sqrt(value)
    power_value = pow(value, 2)
    circle_constant = pi
    print(f"Square root of {value}: {square_root}")
    print(f"Value squared: {power_value}")
    print(f"Pi constant: {circle_constant}")


demonstrate_math_utilities(16.0)
```

25. Parameters

```
def add_two_numbers(num1: float, num2: float) -> float:
    if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
        raise TypeError("Both inputs must be numeric values.")
    return num1 + num2


result = add_two_numbers(5, 3)
print(f"Result of add(5, 3): {result}")
```

26. Multiple Parameters

```
def calculate_rectangle_area(length: float, width: float) -> float:
    if length <= 0 or width <= 0:
        raise ValueError("Dimensions must be positive numeric values.")
    return length * width


rect_area = calculate_rectangle_area(5, 3)
print(f"Area of rectangle (5x3): {rect_area}")
```

27. Len Function

```
def get_string_length(text: str):
    if not isinstance(text, str):
        raise TypeError("Input must be a string text data type.")
    length = len(text)
    print(f"Length of text: {length}")


get_string_length("Cybersecurity Expert")
```

28. Write to File

```
def write_greeting_to_file():
    filename = "greeting.txt"
    with open(filename, "w", encoding="utf-8") as file:
        file.write("Hello World")
    print(f"Success: Content written to {filename}")


write_greeting_to_file()
```

29. Read from File

```
def read_greeting_from_file():
    filename = "greeting.txt"
    try:
        with open(filename, "r", encoding="utf-8") as file:
            content = file.read()
        print("File Contents:")
        print(content)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")


read_greeting_from_file()
```

30. Basic Try-Except

```
def safe_divide(a: float, b: float):
    try:
        res = a / b
        print(f"Division Result: {res}")
    except ZeroDivisionError:
        print("Error: Division by zero is undefined.")
    except TypeError:
        print("Error: Inputs must be numerical values.")


safe_divide(10, 2)
safe_divide(10, 0)
```

31. Create List

```
def display_shopping_cart():
    cart_items = [100, 250, 75]
    print("Shopping Cart Contents:", cart_items)


display_shopping_cart()
```

32. Append to List

```
def add_expense_item(expenses_list: list, amount: float):
    if not isinstance(amount, (int, float)) or amount < 0:
        raise ValueError("Expense amount must be a non-negative number.")
    expenses_list.append(amount)
    print("Updated Expenses List:", expenses_list)


current_expenses = [1200.50, 450.00]
add_expense_item(current_expenses, 320.75)
```

33. Update Dictionary

```
def merge_employee_records(dict1: dict, dict2: dict):
    if not isinstance(dict1, dict) or not isinstance(dict2, dict):
        raise TypeError("Both inputs must be dictionaries.")
    dict1.update(dict2)
```

```
    print("Updated Employee Records:", dict1)


primary_record = {"id": "E101", "name": "Raj",
"role": "Developer"}

secondary_record = {"role": "Senior
Developer", "department": "Engineering"}

merge_employee_records(primary_record,
secondary_record)
```

34. Nested Dictionary

```
def
fetch_nested_employee_salary(dept_data:
dict, department: str, employee_name: str):

    if department not in dept_data:

        print(f"Error: Department '{department}'
does not exist.")

        return

    if employee_name not in
dept_data[department]:

        print(f"Error: Employee
'{employee_name}' not found in
'{department}' department.")

        return


    salary =
dept_data[department][employee_name].get
("salary")

    print(f"Salary of {employee_name}
({department}): {salary}")


company_departments = {

    "Engineering": {

        "Shubh": {"salary": 75000, "role":
"Backend Engineer"},

        "Aman": {"salary": 90000, "role":
"Security Architect"}

    },

    "Marketing": {

        "Neha": {"salary": 60000, "role": "SEO
Specialist"}

    }

}

fetch_nested_employee_salary(company_dep
artments, "Engineering", "Shubh")
```

35. Create Tuple

```
def display_fixed_coordinates(coord_tuple:
tuple):

    if not isinstance(coord_tuple, tuple) or
len(coord_tuple) != 2:

        raise ValueError("Invalid tuple structure
for GPS coordinate storage.")

    print(f"Latitude: {coord_tuple[0]},
Longitude: {coord_tuple[1]}")


fixed_location = (13.0827, 80.2707)

display_fixed_coordinates(fixed_location)
```

36. Set Intersection

```
def find_shared_skills(set_a: set, set_b: set):

    if not isinstance(set_a, set) or not
isinstance(set_b, set):

        raise TypeError("Inputs must be set
structures.")

    common = set_a & set_b

    print("Common Skills Found:", common)
```

```
dev_skills = {"Python", "C++", "Linux", "Git"}
sec_skills = {"Linux", "Wireshark", "Python", "Metasploit"}
find_shared_skills(dev_skills, sec_skills)
```

37. Multiple Instances

```
class Employee:
    def __init__(self, name: str, salary: int):
        self.name = name
        self.salary = salary

    def display_info(self):
        print(f"Employee Name: {self.name}, Salary: {self.salary}")

emp1 = Employee("Vikram", 65000)
emp2 = Employee("Kavitha", 85000)

emp_roster = [emp1, emp2]
for emp in emp_roster:
    emp.display_info()
```

38. Method Chaining

```
class ChainedEmployee:
    def __init__(self, name: str):
        self.name = name
        self.salary = 0

    def set_salary(self, amount: int):
        if amount < 0:
            raise ValueError("Salary cannot be a negative amount.")
        self.salary = amount
        return self

    def apply_raise(self, percentage: float):
        if percentage < 0:
            raise ValueError("Raise percentage cannot be negative.")
        self.salary = int(self.salary * (1 + percentage / 100))
        return self

    def display_final_salary(self):
        print(f"Final Salary for {self.name}: {self.salary}")
        return self

chained_emp = ChainedEmployee("Rohan")
chained_emp.set_salary(50000).apply_raise(10).display_final_salary()
```

39. Polymorphism

```
class Worker:
    def __init__(self, name: str):
        self.name = name

    def work(self):
        return f"{self.name} is performing basic processing duties."
```

```
class SoftwareEngineer(Worker):
    def work(self):
        return f"{self.name} is implementing and testing software modules."

class SecurityAnalyst(Worker):
    def work(self):
        return f"{self.name} is performing penetration vulnerability scans."

workers_list = [SoftwareEngineer("Amit"), SecurityAnalyst("Priya"), Worker("Suresh")]
for worker in workers_list:
    print(worker.work())
```

40. Class Methods

```
class FactoryEmployee:
    def __init__(self, name: str, salary: int):
        self.name = name
        self.salary = salary

    @classmethod
    def from_string(cls, data_string: str):
        parts = data_string.split(",")
        if len(parts) != 2:
            raise ValueError()
        name = parts[0].strip()
        salary = int(parts[1].strip())
        return cls(name, salary)

    def display_details(self):
        print(f"{self.name} {self.salary}")

new_factory_emp = FactoryEmployee.from_string("Shubh, 75000")
new_factory_emp.display_details()
```

41. Employee Management System

```
import json

class SystemEmployee:
    def __init__(self, emp_id: str, name: str, role: str):
        self.emp_id = emp_id
        self.name = name
        self.role = role

    def __str__(self):
        return f"{self.emp_id} {self.name} {self.role}"

class ManagementSystem:
    def __init__(self):
        self.employees = {}

    def add_employee(self, emp: SystemEmployee):
        self.employees[emp.emp_id] = {"name": emp.name, "role": emp.role}

    def save_to_file(self, filename="emps.json"):
        with open(filename, "w") as file:
```

```
        json.dump(self.employees, file)

    def load_from_file(self, filename="emps.json"):
        try:
            with open(filename, "r") as file:
                self.employees = json.load(file)
        except:
            self.employees = {}

    def print_all(self):
        for k, v in self.employees.items():
            print(k, v["name"], v["role"])

ems = ManagementSystem()
ems.add_employee(SystemEmployee("E001", "Arjun", "Security Lead"))
ems.add_employee(SystemEmployee("E002", "Deepa", "Data Architect"))
ems.save_to_file()

new_ems = ManagementSystem()
new_ems.load_from_file()
new_ems.print_all()
```

42. Data Analysis Pipeline

```
import statistics

def setup_mock_sales_file():
    with open("sales.txt", "w") as f:
        f.write("1500\n2300\ntext_error\n3400\n1200\n4500\n")

def process_sales_pipeline(filename="sales.txt"):
    data = []
    try:
        with open(filename, "r") as f:
            for line in f:
                try:
                    data.append(float(line.strip()))
                except:
                    pass
    except:
        return
    if not data:
        return
    print(len(data), statistics.mean(data), statistics.median(data))

setup_mock_sales_file()
process_sales_pipeline()
```

43. Configuration Manager

```
import configparser

def setup_mock_ini():
    with open("db.ini", "w") as f:
        f.write("[Database]\nhost=localhost\nport=5432\nuser=root\n")

class Config:
    def __init__(self, filename):
```

```
        self.filename = filename
        self.parser = configparser.ConfigParser()

class DatabaseConfig(Config):
    def load_settings(self):
        self.parser.read(self.filename)
        return dict(self.parser["Database"])

setup_mock_ini()
db = DatabaseConfig("db.ini")
print(db.load_settings())
```

44. CSV Data Processor

```
import csv

def setup_mock_csv():
    with open("employees.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["name", "salary", "department"])
        w.writerow(["A", "45000", "QA"])
        w.writerow(["B", "62000", "Eng"])

def process_employee_csv(filename="employees.csv"):
    data = []
    with open(filename, "r") as f:
        r = csv.DictReader(f)
        for row in r:
            data.append({"salary": float(row["salary"])})
    filtered = [x for x in data if x["salary"] > 50000]
    if filtered:
        print(sum(x["salary"] for x in filtered)/len(filtered))

setup_mock_csv()
process_employee_csv()
```

45. Expense Tracker

```
from datetime import datetime

def setup_mock_expenses_csv():
    with open("expenses.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["date", "amount", "category"])
        w.writerow(["2026-06-01", "1000", "Food"])

def analyze_monthly_expenses(filename="expenses.csv"):
    total = {}
    now = datetime.now().strftime("%Y-%m")
    with open(filename) as f:
        r = csv.DictReader(f)
        for row in r:
            if row["date"].startswith(now):
                total[row["category"]] = total.get(row["category"],0)+float(row["amount"])
    print(total)
```

```
setup_mock_expenses_csv()
analyze_monthly_expenses()
```

46. API Response Handler

```
import requests

def fetch_weather_metrics():
    r = requests.get("https://api.open-meteo.com/v1/forecast?latitude=13.0827&longitude=80.2707&current_weather=true")
    d = r.json()
    c = d.get("current_weather",{})
    print(c.get("temperature"), c.get("windspeed"))

fetch_weather_metrics()
```

47. Complete Calculator Program

```
def calculate(a,b,op):
    if op=="+": return a+b
    if op=="-": return a-b
    if op=="*": return a*b
    if op=="/": return a/b
```

48. Shopping Cart System

```
class CartItem:
    def __init__(self,n,p,q):
        self.n=n; self.p=p; self.q=q

class ShoppingCart:
    def __init__(self):
        self.items=[]
    def add_item(self,i):
        self.items.append(i)
    def total(self):
        return sum(i.p*i.q for i in self.items)

cart=ShoppingCart()
cart.add_item(CartItem("A",100,2))
print(cart.total())
```

49. Temperature Converter GUI

```
class TemperatureConverter:
    @staticmethod
    def c_to_f(c): return (c*9/5)+32

print(TemperatureConverter.c_to_f(0))
```

50. Backup Utility

```
import os, shutil

def backup():
    os.makedirs("src",exist_ok=True)
    os.makedirs("bak",exist_ok=True)
    with open("src/a.txt","w") as f: f.write("x")
    for f in os.listdir("src"):
        shutil.copy("src/"+f,"bak/"+f)

backup()
```

51. URL Shortener

```
import hashlib
```

```
class URLShortener:
    def __init__(self):
        self.db={}
    def shorten(self,u):
        h=hashlib.md5(u.encode()).hexdigest()[:6]
        self.db[h]=u
        return h
    def resolve(self,h):
        return self.db[h]

s=URLShortener()
h=s.shorten("https://x.com")
print(s.resolve(h))
```

52. Gradebook System

```
class GradebookSystem:
    def __init__(self):
        self.d={}
    def add(self,n,g):
        self.d.setdefault(n,[]).append(g)
    def avg(self):
        all=[]
        for v in self.d.values(): all+=v
        return sum(all)/len(all)

g=GradebookSystem()
g.add("A",90)
print(g.avg())
```

53. Task Scheduler

```
class Task:
    def __init__(self,d,dt,p):
        self.d=d;
self.dt=datetime.strptime(dt,"%Y-%m-%d");
self.p=p

class TaskScheduler:
    def __init__(self):
        self.t=[]
    def add(self,x):
        self.t.append(x)
    def sort(self):
        return sorted(self.t,key=lambda x:(x.dt,-
x.p))

ts=TaskScheduler()
ts.add(Task("A","2026-06-10",1))
print(ts.sort())
```

54. Inventory Manager

```
class Product:
    def __init__(self,i,n,s):
        self.i=i; self.n=n; self.s=s

class WarehouseInventoryManager:
    def __init__(self):
        self.inv={}
    def add(self,p):
        self.inv[p.i]=p
    def low(self):
        return [p.n for p in self.inv.values() if
p.s<5]
```

```
w=WarehouseInventoryManager()
w.add(Product("1","Milk",2))
print(w.low())
```

55. Budget Planner

```
import matplotlib.pyplot as plt

class CategoryExpenseBudget:
    def __init__(self,n,l):
        self.n=n; self.l=l; self.s=0
    def add(self,a):
        self.s+=a

def chart(m):
    labels=[x.n for x in m.values()]
    sizes=[x.s for x in m.values()]
    plt.pie(sizes,labels=labels)

b={"Food":CategoryExpenseBudget("Food",50
00)}
b["Food"].add(2000)
chart(b)
```